

9 Running Times for Arithmetic

In Chapter 5 we stated that we can only assume that basic arithmetic operations run in constant time when the numbers we are working with are relatively small, i.e. when the number of digits is $O(\log n)$.

However, very large numbers occur with some frequency in computer science. Modern cryptography in particular is built on the inefficiency of performing certain operations on large numbers, and many cryptographic algorithms involve multiplying numbers with thousands of digits.

In this chapter we will examine in detail the runtime of performing arithmetic on large numbers. In Section 9.1 we will clarify the encoding that we are using for large numbers and look at the runtime for comparing them. In Section 9.2 we will analyze the runtime for addition (and subtraction). In the remaining three sections we will analyze three different algorithms for multiplication. First we will discuss the ordinary grade school algorithm in Section 9.3. Then we will consider a divide and conquer based approach in Section 9.4. Finally, we will look at another divide and conquer multiplication algorithm in Section 9.5 and discuss the fastest known algorithms for multiplication.

9.1 Comparison

Let's start by considering the runtime to compare two large integers. Previously we've been informal about how we encode numbers as strings, but for this chapter it will be important to be precise. We will view a non-negative integer as a string of decimal digits. Thus, our alphabet is $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ and for each $n \in \mathbb{Z}^+$, $\langle n \rangle$ is the base-10 representation of n . Thus, a string $x = x_1x_2 \dots x_n$ represents the number $\sum_{i=0}^{n-1} 10^i x_{n-i}$. For example, the string "123" represents $3 \cdot 10^0 + 2 \cdot 10^1 + 1 \cdot 10^2 = 3 + 20 + 100$. Note that the strings "0123" and "00123" (along with infinitely many strings with more leading zeros) also represent this number. When we write $\langle n \rangle$, however, we mean the representation with no leading 0s. While we will not bother ensuring that every algorithm in this chapter returns an encoding with no leading 0s, we can guarantee this in linear time with an extra step to remove leading 0s before returning if it is important. We represent the number zero with a single 0 (which we do not consider a leading 0).

All the algorithms we discuss in this chapter would work equally well in another base (e.g. binary instead of base-10) with the same asymptotic runtime, but we choose to work in base-10 since it is the most familiar to most of us.

To get a feel for algorithms working with large numbers, we begin by looking at the time to compare two non-negative integers. We can start by comparing the lengths of the numbers. If one of the numbers has fewer digits than the other, then that number is certainly smaller. If the numbers have the same number of digits, we compare their digits from first to last (most significant to least significant), looking for the first place where they differ. In this first position of difference, whichever number has the larger digit is the larger number. If the two numbers have the same digit at every position, then they are equal.

This approach to comparing large numbers is described in detail by the following algorithm `LESTHAN`.

<pre> LESTHAN(x, y) Remove any leading 0s from x and y If $\text{len}(x) < \text{len}(y)$: Return True If $\text{len}(y) < \text{len}(x)$: Return False For i from 1 to $\text{len}(x)$: If $x_i < y_i$: Return True If $y_i < x_i$: Return False Return False </pre>
--

Theorem 47. *For all non-negative integers x and y , `LESTHAN`(x, y) runs in time $O(\text{len}(x) + \text{len}(y))$.*

Proof. Removing leading zeros from x and y takes time $O(\text{len}(x) + \text{len}(y))$. We note that $\text{len}(x)$ and $\text{len}(y)$ are values less than the length of the input, so they fit in integer RAM model variables and we can compare them in constant time. The for loop runs at most $\text{len}(x) \in O(\text{len}(x) + \text{len}(y))$ times. Inside the loop we are comparing single digits, which have value at most 9, so each iteration of the loop takes constant time. Thus, the overall runtime of the algorithm is $O(\text{len}(x) + \text{len}(y))$. $\square$

We note that our upper bound of $O(\text{len}(x) + \text{len}(y))$ is tight (i.e. the runtime of $\text{LESTHAN}(x, y)$ is $\Theta(\text{len}(x) + \text{len}(y))$) when x and y have the same length. In the worst case, x and y are equal, so the algorithm has to look at all of the digits of each number which takes $\Omega(\text{len}(x) + \text{len}(y))$ time. Furthermore, this runtime is optimal, *any* (deterministic) algorithm for this problem must look at all $\text{len}(x) + \text{len}(y)$ digits of x and y in the worst case, because if there is some digit that our algorithm doesn't look at, then the algorithm cannot distinguish the case where x and y are the same from the case where they differ at exactly that digit.

We note that if we have an algorithm for LESTHAN , we don't have to separately describe and analyze algorithms for related comparisons like greater than and equal to. Instead, we can *reduce* these operators to less than, as shown below:

$\text{GREATERTHAN}(x, y)$ Return $\text{LESTHAN}(y, x)$

$\text{EQUALTO}(x, y)$ Return $\neg \text{LESTHAN}(x, y) \wedge \neg \text{LESTHAN}(y, x)$

9.2 Addition

Now let's look at the runtime to add two non-negative integers. Here again the best algorithm is one that you are already familiar with from grade school. We add pairs of digits, going from the right side of the inputs to the left. When the sum of two digits will not fit in a single digit, we use a "carry" to incorporate the tens digits of this sum into our addition of the pair of the digits to the left (which represent values ten times as large). This algorithm is specified by the pseudocode below.

$\text{ADD}(x, y)$ $m \leftarrow \text{len}(x)$ $n \leftarrow \text{len}(y)$ $\ell \leftarrow \max(m, n)$ Pad x and y with 0s from the front until they both have length ℓ $r \leftarrow$ a string of ℓ zeros carry $\leftarrow 0$ For i from ℓ down to 1: $r_i \leftarrow (x_i + y_i + \text{carry}) \bmod 10$ carry $\leftarrow \lfloor (x_i + y_i + \text{carry}) / 10 \rfloor$ If carry > 0 : Prepend carry to the front of r Return r

Theorem 48. For all non-negative integers x and y , $\text{ADD}(x, y)$ runs in time $O(m + n)$ where m is the length of x and n is the length of y .

Proof. Once again, the length of x and y fit in RAM model integers, so we can compute their max in constant time. Padding x and y to have length ℓ and prepending carry at the end can be done in $O(m + n)$ time since $\ell \leq m + n$. Our for loop runs $O(m + n)$ times for the same reason, and each iteration of the for loop takes constant time since x_i , y_i and carry are all at most 9, so we can do arithmetic on them in constant time. Thus, the algorithm runs in time $O(m + n)$. $\square$

As with LESTHAN , this runtime is the best that we can hope for. The optimality of ADD follows from the fact that any algorithm for adding two integers, no matter how clever, must at least *write down the answer*. For non-negative integers x and y , $x + y$ has at least $\max(\text{len}(x), \text{len}(y)) \in \Omega(m + n)$

digits, so even if we compute each character of the output in constant time, just writing it down already takes $\Omega(m + n)$ time, and thus no algorithm for addition can be faster than this.

We note that we could create an $O(m + n)$ time algorithm SUBTRACT in very similar fashion. The details are left as an exercise to the student.

9.3 Multiplication

For the rest of the chapter, we will consider the problem of multiplying two large positive integers. Once again we will start by considering a typical grade school algorithm (I will present the algorithm that I learned as a child in the US. There are several other well-known multiplication algorithms with equivalent runtimes. Depending on where you went to school you may have learned a slightly different algorithm).

The idea of long multiplication is to break down the problem of multiplying an m digit number with an n digit one into mn single digit multiplications plus n additions. The algorithm multiplies each digit of y with all of x by going from right to left, multiplying digits and keeping track of a carry, similarly to the ADD algorithm. Then the algorithm uses the fact that

$$x \cdot y = \sum_{i=1}^n x \cdot y_i \cdot 10^{\text{len}(y)-i}$$

```

LONGMULTIPLY( $x, y$ )
  If  $\text{len}(x) < \text{len}(y)$ :
    Swap  $x$  and  $y$ 
   $m \leftarrow \text{len}(x)$ 
   $n \leftarrow \text{len}(y)$ 
   $r \leftarrow$  a string of  $m$  zeros
  For  $i$  from  $n$  down to 1:
     $z \leftarrow$  a string of  $m$  zeros
    carry  $\leftarrow 0$ 
    For  $j$  from  $m$  down to 1:
       $z_j \leftarrow (x_j \cdot y_i + \text{carry}) \bmod 10$ 
      carry  $\leftarrow \lfloor (x_j \cdot y_i + \text{carry}) / 10 \rfloor$ 
    If carry  $> 0$ :
      Prepend carry to the front of  $z$ 
    Append  $n - i$  zeros to the end of  $z$ 
     $r \leftarrow \text{ADD}(r, z)$ 
  Return  $r$ 

```

Theorem 49. For all non-negative integers x and y , LONGMULTIPLY(x, y) runs in time $O(mn)$ where m is the length of x and n is the length of y .

Proof. The work outside of the for loops can be done in $O(m)$ time. We note that the first two lines guarantee that $m \geq n$. The outer for loop runs n times.

The inner for loop runs m times and does $O(1)$ work per iteration (since the numbers that we are performing arithmetic with inside this loop are constant size). Thus the total work of the inner for loop is $O(m)$ each iteration. Outside of the for loop, creating z and carry, prepending carry to z and appending $n - i$ zeros can all be done in $O(m)$ time. By Theorem 48 we know that the call to ADD runs in time $O(m + \text{len}(r))$. The length of r is initially m and increases by at most 1 with each iteration of the outer for loop, so $\text{len}(r) \in O(m + n) = O(m)$ (since $m \geq n$). Thus, the runtime of the call to ADD is $O(m)$ and therefore the total runtime of each iteration of the outer for loop is $O(m)$.

Since the outer for loop runs $O(n)$ times and does $O(m)$ work each time, the total runtime for the loop is $O(mn)$ and thus this is the total runtime of the algorithm. $\square$

Note that if y has constant length (e.g. if we're multiplying x by a constant, like 411), then the runtime of this algorithm is $O(m)$, which is the best that we can hope for since, as in the case of ADD, just writing down the answer to this multiplication problem takes $\Omega(m)$ time.

If the two numbers about the same length, however (i.e. if $m = n$), then this algorithm runs in $O(n^2)$, which, perhaps surprisingly, is far from optimal. In the next section we'll look at a new approach for multiplication using a divide and conquer strategy.

9.4 Multiplication with Divide and Conquer

Our divide and conquer multiplication algorithm is based on splitting each of the numbers in half and multiplying the halves. In particular, we note that if x and y are n -digit numbers for some even integer n , we can express x and y as

$$x = 10^{n/2}a + b, \text{ and } y = 10^{n/2}c + d,$$

for some non-negative integers a, b, c, d .

Then,

$$x \cdot y = (10^{n/2}a + b)(10^{n/2}c + d) = 10^n ac + 10^{n/2}ad + 10^{n/2}bc + bd.$$

Multiplying by powers of 10 is very easy, since we can just add 0s to the end of the number. Thus, if we can compute the four smaller products ac, ad, bc, bd , we can compute xy using only additions. The full pseudocode of the algorithm is given below:

```

DC-MULTIPLY( $x, y$ )
   $n \leftarrow \max(\text{len}(x), \text{len}(y))$ 
  Pad  $x$  or  $y$  with leading zeros so that they both have length  $n$ 
  If  $n = 1$ :
    Return  $x_1 \cdot y_1$ 
   $a \leftarrow x_1 x_2 \dots x_{\lfloor n/2 \rfloor}$ 
   $b \leftarrow x_{\lfloor n/2 \rfloor + 1} \dots x_{n-1} x_n$ 
   $c \leftarrow y_1 y_2 \dots y_{\lfloor n/2 \rfloor}$ 
   $d \leftarrow y_{\lfloor n/2 \rfloor + 1} \dots y_{n-1} y_n$ 
   $ac \leftarrow \text{DC-MULTIPLY}(a, c)$ 
   $ad \leftarrow \text{DC-MULTIPLY}(a, d)$ 
   $bc \leftarrow \text{DC-MULTIPLY}(b, c)$ 
   $bd \leftarrow \text{DC-MULTIPLY}(b, d)$ 
  Append  $2(n - \lfloor n/2 \rfloor)$  zeros to the end of  $ac$ 
  Append  $n - \lfloor n/2 \rfloor$  zeros to the end of  $ad$ 
  Append  $n - \lfloor n/2 \rfloor$  zeros to the end of  $bc$ 
  Return ADD(ADD( $ac, ad$ ), ADD( $bc, bd$ ))

```

Theorem 50. *If x and y are n -digit non-negative integers then $\text{DC-MULTIPLY}(x, y)$ runs in time $O(n^2)$.*

Proof. All of the steps outside of the recursion can be performed in $O(n)$ time. The algorithm performs 4 recursive calls to DC-MULTIPLY , and each recursive call is on inputs of length $n/2$ (ignoring floors and ceilings). Thus, the runtime $T(n)$ of DC-MULTIPLY satisfies the recurrence:

$$T(n) = 4T(n/2) + O(n).$$

By the master theorem, this recurrence solves to $T(n) \in \Theta(n^2)$. □

If you are paying attention, you'll note that the runtime bound that we proved for DC-MULTIPLY in Theorem 50 is the same as the bound that we proved for LONGMULTIPLY in Theorem 49 (specialized to the case where $m = n$). Divide and conquer didn't help at all! Was this whole algorithmic technique a scam?

It turns out the divide and conquer technique on its own is not enough to speed up multiplication. We need one more clever idea, which we'll see in the next section.

9.5 Karatsuba's Algorithm

In the mid 20th century it was commonly believed that no multiplication algorithm could do significantly better than LONGMULTIPLY. In 1960, prominent mathematician Andrey Kolmogorov held a seminar where he formally conjectured that multiplying two n -digit integers required $\Omega(n^2)$ time. One of the students attending the seminar was Anatoly Karatsuba who, less than a week later, disproved the conjecture by presenting the algorithm we will discuss in this section.

The basic structure of KARATSUBAMULTIPLY is the same as DC-MULTIPLY. We divide x into two halves, a and b , and divide y into two halves c and d . Karatsuba's core insight is that in order to compute the product

$$x \cdot y = (10^{n/2}a + b)(10^{n/2}c + d) = 10^n ac + 10^{n/2}ad + 10^{n/2}bc + bd$$

we do not need to perform four multiplications. We can rewrite

$$10^n ac + 10^{n/2}ad + 10^{n/2}bc + bd = 10^n ac + 10^{n/2}(ad + bc) + bd = 10^n ac + 10^{n/2}((a+b)(c+d) - ac - bd) + bd.$$

Thus, instead of computing the two products ad and bc , we can compute the single product $(a+b)(c+d)$ and then subtract out the ac and bd that we already needed to compute in order to get $ad + bc$. This trick allows us to reduce the 4 recursive calls of DC-MULTIPLY to 3, at the cost of a few extra addition and subtractions. This turns out to make a big difference in the runtime of the algorithm.

```

KARATSUBAMULTIPLY( $x, y$ )
 $n \leftarrow \max(\text{len}(x), \text{len}(y))$ 
Pad  $x$  or  $y$  with leading zeros so that they both have length  $n$ 
If  $n = 1$ :
    Return  $x_1 \cdot y_1$ 
 $a \leftarrow x_1 x_2 \dots x_{\lfloor n/2 \rfloor}$ 
 $b \leftarrow x_{\lfloor n/2 \rfloor + 1} \dots x_{n-1} x_n$ 
 $c \leftarrow y_1 y_2 \dots y_{\lfloor n/2 \rfloor}$ 
 $d \leftarrow y_{\lfloor n/2 \rfloor + 1} \dots y_{n-1} y_n$ 
 $ac \leftarrow \text{KARATSUBAMULTIPLY}(a, c)$ 
 $bd \leftarrow \text{KARATSUBAMULTIPLY}(b, d)$ 
//  $ad, bc \leftarrow (a + b)(c + d) - ac - bd$ 
 $ad\_bc \leftarrow \text{SUBTRACT}(\text{SUBTRACT}(\text{KARATSUBAMULTIPLY}(\text{ADD}(a, b), \text{ADD}(c, d)), ac), bd)$ 
Append  $2(n - \lfloor n/2 \rfloor)$  zeros to the end of  $ac$ 
Append  $n - \lfloor n/2 \rfloor$  zeros to the end of  $ad\_bc$ 
Return  $\text{ADD}(\text{ADD}(ac, ad\_bc), bd)$ 

```

Theorem 51. *If x and y are n -digit non-negative integers then $\text{KARATSUBAMULTIPLY}(x, y)$ runs in time $O(n^{\log_2 3}) \approx O(n^{1.58})$.*

Proof. Once again, all of the steps outside of the recursion can be performed in $O(n)$ time (all of the numbers we are passing to ADD or SUBTRACT have $O(n)$ digits). There are three recursive calls, and each recursive call is on an input of size $n/2$ (we are safely ignoring floors and ceilings as well as the potential 1 extra digit produced by the calls to $\text{ADD}(a, b)$ and $\text{ADD}(c, d)$). Thus, the runtime $T(n)$ of KARATSUBAMULTIPLY satisfies the recurrence:

$$T(n) = 3T(n/2) + O(n).$$

By the master theorem, this recurrence solves to $T(n) = \Theta(n^{\log_2 3})$, substantially improving on the runtime of LONGMULTIPLY. $\square$

Karatsuba's algorithm is not in fact the fastest known algorithm for multiplication.

- Only a few years after Karatsuba's discovery, Andrei Toom and Stephen Cook developed a generalization of Karatsuba's algorithm which splits a number into k parts to achieve a runtime of $O(n^{\log(2k-1)/\log k})$ for any integer $k \geq 2$ (the hidden constant in the big-O expression becomes worse rapidly as k grows, however).

- In 1971, Arnold Schönhage and Volker Strassen developed an algorithm based on the *Fast Fourier Transform* (which is beyond the scope of this course) that multiplies two n digit non-negative integers in $O(n \log n \log \log n)$ time. This was the fastest multiplication algorithm for decades.
- In 2019, David Harvey and Joris van der Hoeven developed an $O(n \log n)$ time algorithm for multiplying two n -digit integers. This is conjectured to be the optimal runtime for multiplication, but we still don't have a better lower bound than $\Omega(n)$ (which comes from the time needed to write down the output).

In future chapters, we will generally avoid worrying about the encodings of integers, and instead rely on the algorithms that we discussed in this chapter. Thus, when we write $x + y$, you should assume that if x and y are too large to fit in integer variables, we are calling ADD and this operation takes $\Theta(\text{len}(x) + \text{len}(y))$ time. When we write $x \cdot y$ you may assume that we can perform the multiplication in time $O(\text{len}(x)\text{len}(y))$ using LONGMULTIPLY or in time $O(n \log n)$, where $n = \max(\text{len}(x), \text{len}(y))$, using the Harvey-van-der-Hoeven algorithm.

9.6 Comprehension Questions

1. Give a recurrence for the following algorithm:

```

DC-ADD( $x, y$ )
 $n \leftarrow \max(\text{len}(x), \text{len}(y))$ 
Pad  $x$  or  $y$  with leading zeros so that they both have length  $n$ 
If  $n = 1$ :
    Return  $x_1 + y_1$ 
 $a \leftarrow x_1 x_2 \dots x_{\lfloor n/2 \rfloor}$ 
 $b \leftarrow x_{\lfloor n/2 \rfloor + 1} \dots x_{n-1} x_n$ 
 $c \leftarrow y_1 y_2 \dots y_{\lfloor n/2 \rfloor}$ 
 $d \leftarrow y_{\lfloor n/2 \rfloor + 1} \dots y_{n-1} y_n$ 
 $ac \leftarrow \text{DC-ADD}(a, c)$ 
 $bd \leftarrow \text{DC-ADD}(b, d)$ 
Append  $n - \lfloor n/2 \rfloor$  zeros to the end of  $ac$ 
Return ADD( $ac, bd$ )

```

2. Suppose that we want to multiply an n digit number by 411. Using the best known algorithm, what is the fastest asymptotic runtime that we can achieve for this multiplication, as a function of n ?
3. True or false? No algorithm for adding two arbitrary n -digit positive integers can run in $o(n)$ time in the worst case.
4. True or false? It has been proven that every algorithm for multiplying two arbitrary n -digit positive integers requires $\Omega(n \log n)$ time in the worst case.

9.7 Exercises

1. Give an $O(m + n)$ time algorithm SUBTRACT(x, y) which computes $x - y$ for large integers x and y . Assume that we have added “−” to our alphabet and can represent negative numbers.
2. A *complex number* is a number of the form $a + bi$, where $a, b \in \mathbb{R}$ and $i := \sqrt{-1}$ is the imaginary unit. The product of two complex numbers $a + bi$ and $c + di$ is

$$(a + bi)(c + di) = ac + adi + bci + bdi^2 = (ac - bd) + (ad + bc)i.$$

Suppose that you have access to subroutines to add and multiply real numbers. Clearly we can calculate this product using 4 real number multiplications: ac, ad, bc , and bd . Give an algorithm to calculate $(a + bi)(c + di)$ using only *three* real number multiplications.

3. Consider the following algorithm whose correctness we proved back in Theorem 15:

```

FACT(x):
  If  $x \leq 1$ :
    Return 1
  Return  $x * \text{FACT}(x - 1)$ 

```

- (a) How many multiplications does FACT perform in the worst case, in terms of $n := \text{len}(x)$?
 - (b) Prove that any algorithm for computing $x!$ must have a runtime which is $\Omega(n10^n)$.
 - (c) What is the asymptotic runtime of FACT, in terms of n , assuming that the multiplications are done with LONGMULTIPLY?
4. Hexadecimal (also known as base-16) is a system for representing numbers where in addition to the ten digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, there are also digits A, B, C, D, E , and F (corresponding to the decimal numbers 10, 11, 12, 13, 14, and 15, respectively), and place values are powers of 16, instead of powers of 10. For example, the hexadecimal value $3AF$ when converted to decimal would become $3 \cdot 16^2 + 10 \cdot 16^1 + 15 \cdot 16^0 = 943$. Design an algorithm which takes as input an n -digit number in hexadecimal (represented as a string) and outputs the same number in base-10. Your algorithm's runtime should be better than $\Theta(n^2)$.
5. Recall that a degree d polynomial in x is an expression of the form $c_0 + c_1x + c_2x^2 + \dots + c_dx^d$, where $c_0, \dots, c_d$ are called the coefficients of the polynomial. Give an algorithm which takes as input two degree d polynomials in x with integer coefficients and outputs their product. For example, given the polynomials $2x^2 + 3x - 4$ and $-3x^2 + 5x - 2$ your algorithm should output $8 - 26x + 23x^2 + x^3 - 6x^4$. Assume that you can perform addition and multiplication of integers in constant time. Your algorithm should run in time $O(d^{1.99})$ or better.